

Specifying and Verifying Future Conditions

Yahui Song^{*}[0000–0002–9760–5895] Darius Foo^[0000–0002–3279–5827]
Wei-Ngan Chin^[0000–0002–9660–5682]

School of Computing, National University of Singapore, Singapore
e0210374@u.nus.edu, dariusf@u.nus.edu, chinwn@nus.edu.sg

Abstract. This paper formalizes *future conditions*, which complement traditional pre- and post-conditions to provide a more comprehensive specification of each function’s *behaviour* and *expectation*. Pre-conditions govern the required states before each function call, while post-conditions define the immediate outcomes (post-states) upon completion. Future conditions extend this paradigm by specifying expected temporal behaviors and states that manifest after the function call has finished, potentially affecting subsequent operations or program states. Together, these three types of conditions form a robust specification mechanism for reasoning about API behaviors across various temporal contexts. However, existing techniques for reasoning about future conditions have three key limitations: inefficient entailment checking, under-approximation of program behaviors, and bounded loop unrolling. To address these challenges, we propose a set of over-approximating Hoare-style forward rules that accommodate future conditions that are processed once per method declaration. Moreover, we propose a novel solution for modelling recursive behaviors via a *bag* of future conditions, which can be heuristically synthesized and verified in the verification system. We formally prove the soundness of our proposal in Coq and use experimental results to demonstrate its effectiveness in detecting non-trivial, real-world API misuses.

Keywords: Future Conditions · Hoare Logic · Linear Temporal Logic · Separation Logic · Coq Proof Assistant

1 Introduction

Pre-conditions and post-conditions are fundamental concepts in formal methods of software engineering, particularly in the context of *design by contract* [11]. Considering function definitions as the smallest software components, classic pre- and post-conditions provide constraints for behaviors before the function call and expected outcomes from the current function execution, respectively. However, they are inflexible in expressing constraints on program behavior after the function call has completed. This limitation becomes particularly apparent when modeling complex API behaviors, where the impact of a function call may extend beyond its immediate execution context. For instance, they struggle to

^{*} Corresponding author

capture requirements such as: “Opening a read-only file should not be followed by any writing operations”; “Memory allocated by *malloc* must be finally freed before exiting the program”; or “If loading a certificate returns an error code, the program must exit in the next step”, etc. These examples illustrate the need for a mechanism to specify and reason about program behavior beyond the immediate scope of a function call, highlighting a limitation in the expressiveness of traditional pre- and post-conditions.

Prior work [15] proposes *future conditions* to express the aforementioned constraints on program behavior after function calls have been completed. When combined with pre- and post-conditions, this triplet-style specification effectively encapsulates a usage protocol for each function and the key APIs involved. For example, the specification for *malloc* is written in Fig. 1¹. Its pre/post-condition state that the input value should be positive and when a pointer is successfully allocated, i.e., $res \neq null$, it triggers an event *malloc*(*res*), where *res* denotes the return value. We use ϵ for empty traces and $_*$ for permitting any traces. Its future-condition enforces that the allocated pointer should be finally freed ($\mathcal{F}$ denotes the temporal operator *finally*), which effectively prevents memory-leak. The *free* function triggers an event *free*(*ptr*) and its future condition ensures that after deallocation, the input pointer cannot be accessed during its lifetime ($\mathcal{G}$ denotes the temporal operator *globally*), which effectively prevents double free or use after free violations.

```

void *malloc (size_t size);
// pre: size>0  $\wedge$   $\_*$ 
// post: (res=null  $\wedge$   $\epsilon$ )  $\vee$  (res $\neq$ null  $\wedge$  malloc(res))
// future: (res=null  $\wedge$   $\_*$ )  $\vee$  (res $\neq$ null  $\wedge$   $\mathcal{F}$  free(res))

void free (void *ptr);
// pre: true
// post: true  $\wedge$  free(ptr)
// future: true  $\wedge$   $\mathcal{G}$  !_ (ptr)

```

Fig. 1. Triplet specifications for *malloc* and *free* APIs, taken from [15]

Future conditions provide a general mechanism for specifying both safety and liveness properties through linear temporal logic (LTL) formulas, including resource usages, null-pointer dereferences or unchecked return values, etc.

However, existing techniques for detecting violations of future conditions suffer from several limitations. First, it uses an inefficient entailment checking strategy, where future conditions must be checked against all subsequent code. For instance, when *malloc* is called, the behaviors of the following code must satisfy *malloc*’s future conditions. This results in unnecessary repeated checking of future conditions. Second, prior work [15] prioritizes bug-finding (no incorrectly flagged safe code) over soundness (no missed violations), which underapproximates program behavior. This involves arbitrarily discarding paths and

¹ The specifications are in the form of $\bigvee(\pi \wedge \theta)$, i.e., a disjunctive set of conjunctions between pure arithmetic constraints (π) and trace constraints (θ).

handling loops via bounded unrolling, risking unsoundness by omitting critical behaviors. Lastly, a key unresolved challenge is the representation of future conditions generated by recursive execution with recursive data structures.

To address the aforementioned challenges, this paper utilizes a set of Hoare-style verification rules to enable a sound reasoning of future conditions. For effective loops working with recursive data structures, we introduce predicates that represent a bag of items for both traces and future conditions, where the commutative law applies. To enhance automation, we deploy a lightweight loop invariant synthesis procedure, the output of which is constructed by such predicates and can be reliably verified by the proposed verification system. Overall, our approach aims to establish a verification framework based on over-approximation, designed to soundly prove the absence of violations of future conditions.

1. We introduce a set of sound forward reasoning rules to propagate future conditions efficiently. These rules employ over-approximation of behaviors, guaranteeing the absence of false negatives (missed violations) when verifying future conditions. The formal definition and proof of soundness for these rules are presented in Theorem 3.
2. To facilitate the forward reasoning process, we formalize a set of *trace inclusion checking* rules and a set of *trace subtraction* rules. We define and prove their soundness in Theorem 1 and Theorem 2, respectively.
3. We utilize predicates to represent collections of traces and future conditions and implement a trace invariant synthesis procedure that generates these predicates, which our verification rules can then reliably verify.
4. We demonstrate the effectiveness of the proposed verification framework through experimental results and case studies. All lemmas and theorems presented have been proven in Coq. Our artifact is publicly available [2].

2 Overview and Motivating Examples

This section outlines the current solution and challenges for reasoning about future conditions, highlighting our contributions through examples.

2.1 The Current Solution

The essence of reasoning future conditions in [15] can be captured by the following rule for function calls, where proof obligations are **highlighted**:

$$\frac{f(\bar{x}) [\Phi_{pre}, \Phi_{post}, \Phi_{future}] \in \mathcal{E} \quad \Phi \sqsubseteq \Phi_{pre} \quad \{\Phi \circ \Phi_{post}\} e \{\Phi_e\} \quad \Phi_e \sqsubseteq \Phi_{future}}{\{\Phi\} f(\bar{x}); e \{\Phi_{post} \circ \Phi_e\}} [FV\text{-}Call\text{-}Inefficient]$$

A traditional Hoare-style verification rule for function calls works roughly as follows: it retrieves the callee f 's specification from the environment $\mathcal{E}$, and if the current program state Φ entails ($\sqsubseteq$) the callee's pre-condition Φ_{pre} , it extends

the program state with the callee’s post-condition. Here “ $\Phi \circ \Phi_{post}$ ”² means to sequentially compose two specifications. Now, having Φ_{future} , [15] extends the rule with one more proof obligation: the behavior of e (where e denotes the rest of code following the call to $f(\bar{x})$), as captured by Φ_e , entails the callee’s future condition, effectively imposing constraints on the code e after the call.

While this solution is correct, it suffers from repeated invocation(s) of entailment checking, as demonstrated in Fig. 2. The example includes multiple memory operations, with a use-after-free (UAF) bug at line 7. According to the specifications in Fig. 1, every call to *malloc* or *free* necessitates checking the subsequent code against the specified future conditions. We mark all triggered trace entailment checks in blue, and the UAF bug is detected when an entailment failure occurs at line 5. This approach introduces redundant checks – for example, events *free(buf2)* (generated from line 5) and *free(buf1)* (generated from line 8) are verified at least twice and four times, respectively. The inefficiency escalates further when future conditions are applied to operations like *strncpy*. In the worst case, the checking process exhibits (informally) quadratic complexity relative to program length. To overcome this limitation, we propose an improved specification method for future conditions that efficiently detects the UAF bug while eliminating redundant re-analysis.

```

1. int main(int argc, char **argv){
2.   char *buf1, *buf2, *buf3;
3.   buf1 = malloc(1);
// malloc(buf2)·free(buf2)·malloc(buf3)·strncpy(buf2)·free(buf1)·free(buf3) ⊆ F(free(buf1))
4.   buf2 = malloc(1);
// free(buf2)·malloc(buf3)·strncpy(buf2)·free(buf1)·free(buf3) ⊆ F(free(buf2))
5.   free(buf2);
// malloc(buf3)·strncpy(buf2)·free(buf1)·free(buf3) ⊈ G(!_ (buf2)) ← Bug Detected!
6.   buf3 = malloc(1);
// strncpy(buf2)·free(buf1)·free(buf3) ⊆ F(free(buf3))
7.   strncpy(buf2, argv[1], 1); //A UAF bug here!
8.   free(buf1);
// free(buf3) ⊆ G(!_ (buf1))
9.   free(buf3);}
// ε ⊆ G(!_ (buf3))

```

Fig. 2. Detecting a UAF bug from CWE-416 [5]

2.2 Towards an Efficient Propagation for Future Conditions

We propose a simplified specification syntax that reduces redundancy by associating future conditions with post-conditions, thereby eliminating unnecessary arithmetic constraints. As shown in Fig. 3, each specification comprises

² The interpretation of $\circ$ depends on the specific logic employed in Φ : for pure arithmetic: $\circ \equiv \wedge$ (logical conjunction); for separation logic: $\circ \equiv *$ (separating conjunction); for linear temporal logic: $\circ \equiv \cdot$ (temporal concatenation), etc.

a precondition (**req** clause), and a *post summary* (**ens** clause) structured as “**req**: π **ens**: $\bigvee (\pi; \theta; F)$ ”. Here, π is the precondition, and each disjunctive case in the post summary includes: a post state pure constraint π , a trace formula θ recording triggered events and an associated future condition F . We use $_*$ for the default future condition, which imposes no constraints on future executions.

The new specifications of *malloc* and *free* is semantically equivalent to their original version shown in Fig. 1. For *strncpy*, it triggers an event involving its first argument. We show in Fig. 4 how the revised specification achieves both conciseness and linear complexity propagation of future conditions.

```

void *malloc (size_t size);
//req: size > 0 ens: (res=null ;  $\epsilon$  ;  $\_*$ )  $\vee$  (res  $\neq$  null ; malloc(res) ;  $\mathcal{F}$ (free(res)))

void free (void *ptr);
//req: true ens: (res=() ; free(ptr) ;  $\mathcal{G}(!_ (ptr))$ )

char *strncpy (char *dest, const char *src, size_t n)
//req: true ens: (res=dest ; strncpy(dest) ;  $\_*$ )

```

Fig. 3. An improved way for specifying future conditions for memory usage APIs

Program states are captured in the form of $\{\bigvee (\pi; \theta; F)\}$. By associating future conditions with program states, we can conjunctively combine different future conditions and compute the next states through *trace subtraction*. Fig. 4 highlights these incremental updates to future conditions. In particular, at line 5, subtracting the event “*free(buf2)*” from “ $\mathcal{F}(\text{free}(\text{buf2}))$ ” yields $_*$, indicating the fulfillment of the deallocation obligation. At line 7, the UAF bug is detected

```

2. char *buf1, *buf2, *buf3;
   {( $\exists$ buf1, buf2, buf3. true ;  $\epsilon$  ;  $\_*$ ) }
3. buf1 = malloc(1);
   {( $\exists$ buf1, buf2, buf3. buf1  $\neq$  null ; malloc(buf1) ;  $\mathcal{F}(\text{free}(\text{buf1}))$ ) }
4. buf2 = malloc(1);
   {( $\exists$ buf1, buf2, buf3. buf1  $\neq$  null  $\wedge$  buf2  $\neq$  null ; malloc(buf1)  $\cdot$  malloc(buf2) ;
      $\mathcal{F}(\text{free}(\text{buf1})) \wedge \mathcal{F}(\text{free}(\text{buf2}))$ ) }
5. free(buf2);
   {( $\exists$ buf1, buf2, buf3. buf1  $\neq$  null  $\wedge$  buf2  $\neq$  null ; malloc(buf1)  $\cdot$  malloc(buf2)  $\cdot$  free(buf2) ;
      $\mathcal{F}(\text{free}(\text{buf1})) \wedge \_*$   $\wedge \mathcal{G}(!_ (buf2))$ ) }
6. buf3 = malloc(1);
   {( $\exists$ buf1, buf2, buf3. buf1  $\neq$  null  $\wedge$  buf2  $\neq$  null  $\wedge$  buf3  $\neq$  null ; malloc(buf1)  $\cdot$  malloc(buf2)
      $\cdot$  free(buf2)  $\cdot$  malloc(buf3) ;  $\mathcal{F}(\text{free}(\text{buf1})) \wedge \mathcal{G}(!_ (buf2)) \wedge \mathcal{F}(\text{free}(\text{buf3}))$ ) }
7. strncpy(buf2, argv[1], 1);
   {( $\exists$ buf1, buf2, buf3. buf1  $\neq$  null  $\wedge$  buf2  $\neq$  null  $\wedge$  buf3  $\neq$  null ; malloc(buf1)  $\cdot$  malloc(buf2)
      $\cdot$  free(buf2)  $\cdot$  malloc(buf3)  $\cdot$  strncpy(buf2) ;  $\mathcal{F}(\text{free}(\text{buf1})) \wedge \perp \wedge \mathcal{F}(\text{free}(\text{buf3}))$ )  $\Leftarrow \mathbf{X}$  }
   FC Violation Found: subtracting “strncpy(buf2)” from “ $\mathcal{G}(!_ (buf2))$ ” leads to false!

```

Fig. 4. Detecting the UAF bug in Fig. 2 in a more efficient way

when we subtract the event “*strncpy(buf2)*” from “ $\mathcal{G}(!_ (buf2))$ ”, producing $\perp$ (*false*) – a contradiction that indicates the invalid usage of “*buf2*”.

Our approach processes future conditions once per method declaration, requiring only a single analysis pass per event. The key idea is to embed future conditions into program states and support logical operations – such as conjunctive obligations and trace subtraction – over these future conditions. We formalize this approach through a set of novel forward rules and prove their soundness in Sec. 4. Next, we illustrate how to specify and verify future conditions for loops.

2.3 Predicates for Bags of Traces and Future Conditions

The program in Fig. 5 creates an array of length n and iteratively invokes *malloc* for each element. The specification for *mallocN*, given in Fig. 6, comprises two predicates: “ $pred_t(B, i)$ ” for triggered traces, and “ $pred_f(B, i)$ ” for the future conditions. Predicates are in the form of “ $\Lambda_i^B(\bigvee(\pi \wedge \theta))$ ”, where i denotes the iterator and B denotes the

```

1 void* mallocN(int n, void **arr,){
2   int i = 0;
3   while (i < n) {
4     arr[i] = malloc(4); i = i+1;}
5   return *arr;}
6
7 void main () {
8   void *arr[5]; mallocN (5, arr);
9   free(arr[0]); /* memory leak */}

```

Fig. 5. Iteratively malloc n times

bag of elements that satisfies the specification $\bigvee(\pi \wedge \theta)$, allowing flexible specification under different arithmetic constraints. Predicates enforce commutativity, discarding trace order sensitivity among each element in the bag.

In Fig. 7, we demonstrate the forward reasoning for *main*. After the function call to *mallocN*(5, *arr*), the predicates are instantiated with the concrete bag, i.e., [0..5). At line 9, when the first element is freed, the future condition predicate

$$\begin{aligned}
 \text{mallocN}(n, \text{arr}) &\equiv \text{req: } \text{length}(\text{arr}) \geq n \\
 &\quad \text{ens: } (\exists i. \text{true} ; \text{pred}_t([0..n), i) ; \text{pred}_f([0..n), i)) \\
 \text{pred}_t(B, i) &\equiv \Lambda_i^B(\text{arr}[i] \neq \text{null} \wedge \text{malloc}(\text{arr}[i])) \vee (\text{arr}[i] = \text{null} \wedge \epsilon) \\
 \text{pred}_f(B, i) &\equiv \Lambda_i^B(\text{arr}[i] \neq \text{null} \wedge \mathcal{F}(\text{free}(\text{arr}[i])))
 \end{aligned}$$

Fig. 6. Specification for *mallocN*

```

8. void *arr[5]; mallocN (5, arr);
   {(\exists arr, i. \text{length}(\text{arr})=5 ; \text{pred}_t([0..5), i) ; \text{pred}_f([0..5), i))}
9. free(arr[0]);
   {(\exists arr, i. \text{length}(\text{arr})=5 ; \text{pred}_t([0..5), i) \cdot \text{free}(\text{arr}[0]) ; \text{pred}_f([1..5), i) \wedge \mathcal{G}(!_ (\text{arr}[0])))}

```

FC Violation Found: empty trace “ ϵ ” does not satisfy the obligation “ $\text{pred}_f([1..5), \text{arr})$ ”!

Fig. 7. Forward reasoning for *main* and detecting the memory leak in Fig. 5

updates to a new bag $[1..5]$, reflecting the removal of $arr[0]$. By the end, the memory leak is detected via a failed entailment checking, that the empty trace is not always permitted in the future condition “ $pred_f([1..5], i)$ ”. To automate this process, we introduce a heuristic-based synthesis for predicate generation, and enable the predicate propagation in the trace subtraction rules.

Remark. Such memory usage violations can also be detected by static analyzers such as Pulse (a memory safety checker in Infer [10]), which relies on separation logic. However, Pulse requires an additional side check upon function exit to account for residual footprints when detecting memory leaks. In contrast, our approach employs a lightweight, general-purpose temporal logic to explicitly define obligations, avoiding the need for implicit checks. This not only simplifies verification but also extends applicability beyond memory safety, enabling the verification of both safety and liveness properties. We showcase the variety of bug types that can be handled by specifying future conditions in Table 2.

3 Target Language and Specifications

We target an imperative, first-order, call-by-value core language, defined in Fig. 8. A program $\mathcal{P}$ comprises a list of function declarations $\overline{Func}$. Here, we use the *overline* to denote a finite list of items, for example, $\overline{x}$ refers to a list of variables, $x_1, \dots, x_n$. Each function has a name f , formal arguments $\overline{x}$, and an expression-oriented body e . Function specifications contains a pre-condition π , and a post summary Δ , which is a set of disjunctive tuples. Each three-element tuple contains: a pure formula π , a trace formula θ , and a future condition F .

We utilize π as the basic logical formula, capturing the Presburger arithmetic conditions on program inputs and local variables. Values include variables and constants ranging from integers, Boolean, unit, null, and $*$ for non-deterministic values. Expressions include sequencing, function calls, conditionals, assignments,

(Program)	$\mathcal{P} ::= \overline{Func}$	(Function Decl.)	$Func ::= f(\overline{x})\{e\}$
(Post Summary)	$\Delta ::= \bigvee (\pi ; \theta ; F)$	(Specification)	$[\mathbf{req}: \pi \ \mathbf{ens}: \Delta]$
(Expressions)	$e ::= v \mid x := e \mid \text{local } x \mid e_1 ; e_2 \mid \text{ev}(\mathbf{A}) \mid$ $\quad \text{if } b \ e_1 \ e_2 \mid \text{while } \pi \ \mathbf{do} \ e \mid f(\overline{x}) \mid \mathbf{assert}_f F$		
(Trace)	$\theta ::= \perp \mid \epsilon \mid \mathbf{A} \mid \theta_1 \vee \theta_2 \mid \theta_1 \wedge \theta_2 \mid \theta_1 \cdot \theta_2 \mid \theta^*$		
(Single Events)	$\mathbf{A} ::= \text{ev}(t) \mid \neg \text{ev}(t) \mid \neg _ (t) \mid _ \mid \text{pred}(B, i)$		
(Future Cond.)	$F ::= \theta$	$\text{pred}(B, i) ::= \Lambda_i^B(\bigvee (\pi \wedge \theta))$	
(Pure)	$\pi ::= T \mid F \mid \text{bop}(t_1, t_2) \mid \pi_1 \wedge \pi_2 \mid \pi_1 \vee \pi_2 \mid \neg \pi \mid \exists x. \pi \mid \forall x. \pi$		
(Terms)	$t ::= v \mid t_1 + t_2 \mid -t$	(Values)	$v ::= x \mid i \mid b \mid () \mid \text{null} \mid *$
(Bag)	$B ::= \emptyset \mid \{v\} \mid B_1 - B_2 \mid B_1 \cup B_2 \mid B_1 \cap B_2$		

Fig. 8. Syntax of the core language and the specification language

while loops, etc. Furthermore, we use “ $ev(A)$ ” for explicitly raising events, and “ $\text{assert}_f F$ ” for asserting constraints for future behaviors.

Traces are regular expressions, comprising *false* ($\perp$); empty traces ϵ ; singleton events A ; sequential concatenations $\theta_1 \cdot \theta_2$; disjunctions $\theta_1 \vee \theta_2$; conjunctions $\theta_1 \wedge \theta_2$; and finite time (zero or many) repetition of a trace, constructed by a Kleene star θ^* . In this paper, we use LTL operators $\mathcal{F}(A)$, $\mathcal{G}(A)$ and $\mathcal{N}(A)$ as short hands for regular expressions “ $(\neg A)^* \cdot A \cdot _$ ”, “ $(A)^*$ ” and “ $_ \cdot A \cdot _$ ”, respectively. Singleton events include: parameterized events $ev(t)$; negated parameterized events $\neg ev(t)$; forbid argument $\neg _ (t)$; wildcards $_$ matching any event; predicates that capture a bag of disjunctive traces. Future conditions constraint temporal behaviors in the future and are essentially trace constructs; we use “ F ” to denote them for clarity and to avoid ambiguity.

We use “ $_$ ” to represent the default future condition, which permits any possible future executions. The Boolean values of T and F are respectively indicated by *true* and *false*. The binary operators *bop* are from $\{<, \leq, =, \geq, >\}$. A term can be a simple value v or simple computations of terms, $t_1 + t_2$ and $-t$. A bag is a set of unique elements, and we use $[0..n]$ as a shorthand for $\{0..(n-1)\}$.

3.1 Instrumented Semantics for the Target Language

To facilitate the soundness proof for the forward rules, we present an instrumented reduction $[s, \rho, F, e] \longrightarrow [s', \rho', F', v]$ for the core language, shown in Fig. 9. Each reduction rule operates on a concrete program state on the left hand

$$\begin{array}{c}
\frac{[\text{OP-Assume}]}{[s, \rho, F', \text{assert}_f F] \longrightarrow [s, \rho, F' \wedge F, ()]} \quad \frac{A = ev(t) \quad F \odot_{lin} A \hookrightarrow_{Pure(s)} F'}{[s, \rho, F, ev(t)] \longrightarrow [s, \rho \uparrow\uparrow [A], F', ()]} [\text{OP-Ev}] \\
\\
\frac{}{[s, \rho, F, v] \longrightarrow [s, \rho, F, v]} [\text{OP-Val}] \quad \frac{}{[s, \rho, F, local\ x] \longrightarrow [s \uparrow\uparrow [x], \rho, F, ()]} [\text{OP-Local}] \\
\\
\frac{[s, \rho, F, e_1] \longrightarrow [s_1, \rho_1, F_1, v_1] \quad [s_1, \rho_1, F_1, e_2] \longrightarrow [s_2, \rho_2, F_2, v_2]}{[s, \rho, F, e_1 ; e_2] \longrightarrow [s_2, \rho_2, F_2, v_2]} [\text{OP-Let}] \quad \frac{x \in dom(s) \quad [s, \rho, F, e] \longrightarrow [s_1, \rho_1, F_1, v]}{[s, \rho, F, x := e] \longrightarrow [s_1[x := v], \rho_1, F_1, ()]} [\text{OP-Assign}] \\
\\
\frac{[s, \rho, F, e_1] \longrightarrow [s', \rho', F', v]}{[s, \rho, F, \text{if true } e_1\ e_2] \longrightarrow [s', \rho', F', v]} [\text{OP-Cond-T}] \quad \frac{[s, \rho, F, e_2] \longrightarrow [s', \rho', F', v]}{[s, \rho, F, \text{if false } e_1\ e_2] \longrightarrow [s', \rho', F', v]} [\text{OP-Cond-F}] \\
\\
\frac{\llbracket \pi \rrbracket_s = true \quad e' = (e ; \text{while } \pi \text{ do } e) \quad [s, \rho, F, e'] \longrightarrow [s', \rho', F', v]}{[s, \rho, F, \text{while } \pi \text{ do } e] \longrightarrow [s', \rho', F', v]} [\text{OP-While-T}] \\
\\
\frac{\llbracket \pi \rrbracket_s = false}{[s, \rho, F, \text{while } \pi \text{ do } e] \longrightarrow [s, \rho, F, ()]} [\text{OP-While-F}] \quad \frac{f(\bar{y})\{e\} \in \mathcal{P} \quad [s, \rho, F, e[\bar{x}/\bar{y}]] \longrightarrow [s', \rho', F', v]}{[s, \rho, F, f(\bar{x})] \longrightarrow [s', \rho', F', v]} [\text{OP-Call}]
\end{array}$$

Fig. 9. Big-step instrumented semantic model for the core language

side, where an expression e is associated with a concrete stack s ; a sequence of events triggered in the course of its execution ρ , and a future condition F . Given Var is a set of variables, and Val denotes all the concrete values, s and ρ are from the following concrete domains: $s \triangleq Var \rightarrow Val$ and $\rho \triangleq list\ A$. The big-step semantics reduce any given program e to a resulting value v , and when the e terminates, the state is transformed from (s, ρ, F) to (s', ρ', F') . We use $\llbracket \pi \rrbracket_s$ to denote that validity of the constraint π with respect to a concrete stack. In particular, when triggering an event $ev(t)$, `[OP-Ev]` subtracts the event from the current future condition. We use $Pure(s)$ to convert a concrete stack into a pure constraint which contains all the equalities between variables and values. Intuitively, $F \odot_{lin} \theta \hookrightarrow_{\pi} F'$ subtracts a trace θ from the given future condition F , resulting in a next state future condition F' . For instance, $\mathcal{F}(free(x)) \odot_{lin} free(x) \hookrightarrow_{true} _*$, and $\mathcal{G}(!_-(x)) \odot_{lin} free(x) \hookrightarrow_{true} \perp$. Trace subtraction is detailed in Sec. 4.3.

3.2 Semantic Model for Logical Assertions

We define the semantic model for program assertions in Fig. 10. Let $s, \rho \models \pi \wedge \theta$ denote the *models* relation, i.e., the concrete stack s and a concrete sequence of events ρ satisfy the logical state π and the temporal specification θ . Here, $\llbracket \rrbracket$ is an empty sequence and $++$ appends two trace sequences. A concrete state s, ρ models a trace predicate if for every element j in the bag B , there exists an instantiated disjunctive case that can be modelled by s, ρ .

$s, \rho \models \pi \wedge \epsilon$	$\Leftrightarrow$	$\rho = []$ and $\llbracket \pi \rrbracket_s = true$
$s, \rho \models \pi \wedge (\theta_1 \vee \theta_2)$	$\Leftrightarrow$	$s, \rho \models \pi \wedge \theta_1$ or $s, \rho \models \pi \wedge \theta_2$
$s, \rho \models \pi \wedge (\theta_1 \wedge \theta_2)$	$\Leftrightarrow$	$s, \rho \models \pi \wedge \theta_1$ and $s, \rho \models \pi \wedge \theta_2$
$s, \rho \models \pi \wedge (\theta_1 \cdot \theta_2)$	$\Leftrightarrow$	exists $\rho_1 \ \rho_2$ such that $\rho_1 ++ \rho_2 = \rho$ and $s, \rho_1 \models \pi \wedge \theta_1, s, \rho_2 \models \pi \wedge \theta_2$
$s, \rho \models \pi \wedge \theta^*$	$\Leftrightarrow$	$s, \rho \models \pi \wedge \epsilon$ or $s, \rho \models \pi \wedge \theta \cdot \theta^*$
$s, \rho \models \pi \wedge ev(t)$	$\Leftrightarrow$	exists t' such that $\rho = [ev(t')]$ and $\llbracket \pi \wedge (t=t') \rrbracket_s = true$
$s, \rho \models \pi \wedge \neg ev(t)$	$\Leftrightarrow$	exists ev', t' such that $\rho = [ev'(t')]$ and either $ev \neq ev'$ or $\llbracket \pi \wedge (t=t') \rrbracket_s = false$
$s, \rho \models \pi \wedge \neg_ (t)$	$\Leftrightarrow$	exists ev, t' such that $\rho = [ev(t')]$ and $\llbracket \pi \wedge (t=t') \rrbracket_s = false$
$s, \rho \models \pi \wedge _$	$\Leftrightarrow$	exists ev, t such that $\rho = [ev(t)]$ and $\llbracket \pi \rrbracket_s = true$
$s, \rho \models \Lambda_i^B(\bigvee(\pi \wedge \theta))$	$\Leftrightarrow$	forall $j \in B$ such that $s, \rho \models (\bigvee(\pi \wedge \theta))[j/i]$

Fig. 10. Semantic model of trace specifications

4 Forward Reasoning

We formalize a set of syntax-directed forward rules in Fig. 11, in the form of Hoare-style triples: $\{\pi\} e \{\Delta\}$, i.e., a shorthand for $\mathcal{E} \vdash \{(\pi; \epsilon; _*)\} e \{\Delta\}$, where $\mathcal{E}$ is an environment mapping from functions to their specifications, ϵ

represents an empty history trace and $_*$ is the default future condition. Under a partial correctness interpretation, which we adopt in this paper, the triple means that if π describes the state before executing e , if e terminates, Δ describes the resulting post-summary.

Rule [FV-Assume-F] interpolates future conditions into program states, thereby constraining subsequent execution behaviors. Rule [FV-Ev] generates one event into the triggered trace, leaving a default future condition, $_*$. Rule [FV-Value] updates the result value in the post state, where we use the reserved variable res to denote the (temporarily) result values, leaving a default future condition. Rule [FV-Seq] reasons about e_1 and e_2 in sequence, and implicitly relies on [FV-Disj] to distribute the disjunctions introduced by e_1 , and [FV-Struct] to propagate traces and future conditions structurally. Rule [FV-Local] introduces an existential variable x into the state. Rule [FV-Cond] computes the post-summaries from both branches by extending the state with b being true and false, respectively; then, it disjunctively unions the results. Rule [FV-Call] retrieves the verified specification of the callee function, checks the entailment between the current state and the callee's pre-condition, then concludes the instantiated post-summary.

Rule [FV-Assign] derives the post-summary Δ_1 for e by introducing a fresh variable r to represent Δ_1 's return values, substituting all occurrences of res with r , and binding x and r in the final post-summary. In [FV-While], the initial state $(\pi; \theta; F)$ serves as a loop invariant, and executing the loop body must re-establish the loop invariant under the same trace and future conditions. We present a trace invariant synthesis procedure in Sec. 4.1.

$$\begin{array}{c}
\frac{}{\{\pi\} \text{assert}_f F \{(\pi; \epsilon; F)\}} \text{[FV-Assume-F]} \quad \frac{}{\{\pi\} \text{ev}(t) \{(\pi; \text{ev}(t); _*)\}} \text{[FV-Ev]} \\
\frac{\pi' = \pi \wedge (res = v)}{\{\pi\} v \{(\pi'; \epsilon; _*)\}} \text{[FV-Value]} \quad \frac{\{\pi\} e_1 \{\Delta_1\} \quad \{\Delta_1\} e_2 \{\Delta_2\}}{\{\pi\} e_1; e_2 \{\Delta_2\}} \text{[FV-Seq]} \\
\frac{\text{[FV-Local]} \quad \text{fresh } x}{\{\pi\} \text{local } x \{(\exists x. \pi; \epsilon; F)\}} \quad \frac{\{\pi \wedge b\} e_1 \{\Delta_1\} \quad \{\pi \wedge \neg b\} e_1 \{\Delta_2\}}{\{\pi\} \text{if } b \text{ } e_1 \text{ } e_2 \{\Delta_1 \vee \Delta_2\}} \text{[FV-Cond]} \\
\frac{\text{fresh } r \quad f(\bar{y})[\text{req: } \pi' \text{ ens: } \Delta] \in \mathcal{E} \quad \pi \leq \pi'[\bar{x}/\bar{y}]}{\{\pi\} f(\bar{x}) \{\exists r. \Delta[\bar{x}/\bar{y}, r/res]\}} \text{[FV-Call]} \\
\frac{\text{[FV-Assign]} \quad \text{fresh } r \quad \{\pi\} e \{\Delta_1\}}{\{\pi\} x := \Delta_1[r/res] \wedge (x=r)} \quad \frac{\text{[FV-While]} \quad \{(\pi \wedge \pi_g; \theta; F)\} e \{(\pi; \theta; F)\}}{\{(\pi; \theta; F)\} \text{while } \pi_g \text{ do } e \{(\pi \wedge \neg \pi_g; \theta; F)\}} \\
\frac{\text{[FV-Disj]} \quad \forall (\pi; \theta; F) \in \Delta. \quad \{(\pi; \theta; F)\} e \{\Delta_i\}}{\{\Delta\} e \{\bigvee \Delta_i\}} \quad \frac{\text{[FV-Struct]} \quad \{\pi\} e \{\Delta\} \quad \forall (\pi'; \theta_1; F_1) \in \Delta. \quad F \odot \theta_1 \hookrightarrow_{\pi'} F'}{\{(\pi; \theta; F)\} e \{\bigvee (\pi'; (\theta \cdot \theta_1); (F' \wedge F_1))\}} \\
\frac{\{\pi_3\} e \{(\pi_4; \theta'; F')\} \quad \pi_1 \leq \pi_3 \quad \pi_4 \leq \pi_2 \quad \theta' \sqsubseteq_{\pi_4} \theta \quad F \sqsubseteq_{\pi_4} F'}{\{\pi_1\} e \{(\pi_2; \theta; F)\}} \text{[FV-Conseq]}
\end{array}$$

Fig. 11. Forward verification and inference rules

Furthermore, [FV-Disj] distributes the disjunctive tuples from the left-hand side and unions their independent reasoning results. Rule [FV-Struct] is a structural rule designed to handle arbitrary tuples from the left-hand side. Given any history traces θ and context future conditions F , [FV-Struct] performs two key operations: it concatenates the extended traces to the history trace ($\theta \cdot \theta_1$); and propagates the subtracted F , represented as $F \ominus \theta_1 \hookrightarrow_\pi F'$, to be part of the post-summary. Intuitively, this rule concludes a conjunctive future condition, i.e., $F' \wedge F_1$, which encompasses both the future condition F_1 obtained by executing e and the future condition propagated from the context F_1 . Lastly, Rule [FV-Conseq] soundly weakens the post-summaries by weakening postconditions (the covariant) and strengthening pre/future-conditions (the contravariants).

Entailment Checking & Trace Subtraction. Pure constraints entailment ($\pi \leq \pi'$) are discharged by the Z3 [6] solver. Trace inclusions ($\theta_1 \sqsubseteq_\pi \theta_2$) are discharged by a term rewriting system (TRS), which is extended from a known solution [4] for solving inequalities between regular expressions, detailed in Sec. 4.2. Trace subtraction from future conditions ($F \ominus \theta \hookrightarrow_\pi F'$) propagates the context future condition concerning specific execution traces, detailed in Sec. 4.3. Overall, our main technical contributions are: extending the TRS to accommodate our new event types, and developing novel trace subtraction rules.

Specification Inference for Function Definition. Given a set of primitive specifications, our goal is to develop a verification system that is *as automated as possible*, where the specification for each function definition is inferred compositionally. The top-level inference process is illustrated in [FV-Func]. The environment $\mathcal{E}$ includes both primitive specifications and the specifications inferred so far. For each function definition $f(\bar{x}) \{e\}$ in the given program, we derive the specification for e using the forward rules in Fig. 11 and then reason the rest of the program with the extended environment $\mathcal{E}'$.

$$\frac{\mathcal{E} \vdash \{\pi'\} e \{\Delta\} \quad \forall(\pi; \theta; F) \in \Delta. \quad \delta(F_\exists) = \text{true} \quad \mathcal{E}' = \mathcal{E} ++ f(\bar{x}) [\text{req: } \pi' \text{ ens: } \Delta] \{e\} \quad \mathcal{E}' \vdash \mathcal{P}}{\mathcal{E} \vdash f(\bar{x}) \{e\}; \mathcal{P}} \text{ [FV-Func]}$$

For each tuple in e 's post-summary Δ , we check whether the empty trace satisfy the future conditions involving existential variables, i.e., $F_\exists$, as the lifetime of existential variables ends when the function concludes. The $F_\exists$ is computed from F by replacing all events related to universally quantified variables – including the formal arguments $\bar{x}$ and the return value res – into wildcards “ $_$ ”, and the resulting future condition thus retains only the constraints on existential variables. For example, in Fig. 5, the memory leak error upon the usage of arr is detected in $main$ instead of $mallocN$, as arr is universally quantified in $mallocN$, but existential quantified in $main$. The Nullable function (cf. Definition 1) returns a Boolean value indicating if θ permits the empty trace.

Definition 1 (Nullable). *Given any sequence θ , we recursively define $\delta(\theta)$:*

$$\begin{aligned} \delta(\theta_1 \cdot \theta_2) &= \delta(\theta_1) \wedge \delta(\theta_2) & \delta(\theta_1 \vee \theta_2) &= \delta(\theta_1) \vee \delta(\theta_2) & \delta(\theta_1 \wedge \theta_2) &= \delta(\theta_1) \wedge \delta(\theta_2) \\ \delta(\Lambda_i^B(\bigvee(\pi \wedge \theta))) &= \exists \theta. (\delta(\theta) = \text{true}) & \delta(\epsilon) &= \delta(\theta^*) = \text{true} & \delta(\perp) &= \delta(A) = \text{false} \end{aligned}$$

4.1 Trace Invariant Synthesis

A key challenge in automating the verification lies in inferring trace invariants – specifically, given a loop expression “**while** π_g **do** e ”, determining the initial state $(\pi; \theta; F)$, deployed in [FV-While]. The trace invariant synthesis process is outlined in Algorithm 1. It takes a loop statement as input and either outputs a valid invariant or triggers a verification failure – indicating that the loop invariants must be provided manually.

At lines 1-2, it derives the specification for e and obtains a fresh iterator i . Then deploys the bag bounds generation function (Definition 2) to produce a set of candidate bags. For each candidate bag B , lines 4-5 construct the trace and future condition predicates respectively. If there exists an invariant that can be re-established after one iteration of the loop body (line 6), it is deemed valid. Otherwise, the synthesis fails and the verification could not proceed due to the absence of trace invariants.

Algorithm 1 Loop Invariant Synthesis

Require: A loop: **while** π_g **do** e

Ensure: A Loop Invariants or Failure

```

1:  $\{\pi'\} \leftarrow e \{ \bigvee(\pi; \theta; F) \}$ 
2: fresh  $i$ 
3: for all  $B \in \mathcal{BB}(\pi_g)$  do
4:    $\theta \leftarrow \Lambda_i^B(\bigvee(\pi \wedge \theta))$ 
5:    $F \leftarrow \Lambda_i^B(\bigvee(\pi \wedge F))$ 
6:   if  $\{(\pi' \wedge \pi_g; \theta; F)\} \in \{(\pi'; \theta; F)\}$ 
7:     then return  $(\pi'; \theta; F)$ 
8: end for
9: return Unknown Loop Invariant

```

Definition 2 (Bag Bounds Generation). *For any loop guard π , we propagate a set of candidate bag bounds using $\mathcal{BB}(\pi)$, for constructing the trace predicates:*

$$\begin{aligned} \mathcal{BB}(t_1 < t_2) &= \{[0..t_1), \dots\} & \mathcal{BB}(\pi_1 \wedge \pi_2) &= \mathcal{BB}(\pi_1) \cup \mathcal{BB}(\pi_2) \\ \mathcal{BB}(t_1 \leq t_2) &= \mathcal{BB}(t_1 < (t_2 + 1)) & \mathcal{BB}(t_1 > t_2) &= \mathcal{BB}(t_2 < t_1) & \mathcal{BB}(t_1 \geq t_2) &= \mathcal{BB}(t_2 \leq t_1) \end{aligned}$$

In Definition 2, the base case is generating the candidate bags for $t_1 < t_2$ while the remaining cases are derived by reduction to the base case. For example, Fig. 12 demonstrates the verification process for the loop in *mallocN* (Fig. 5). At

```

3. while ( $i < n$ ) {
   $\{(\exists i. \text{true}; \text{pred}_t([0..i), i); \text{pred}_f([0..i), i))\}$ 
  4.    $\text{arr}[i] = \text{malloc}(4);$ 
   $\{(\exists i. \text{true}; \text{pred}_t([0..i+1), i); \text{pred}_f([0..i+1), i))\}$ 
  5.    $i = i+1;$ 
   $\{(\exists i. \text{true}; \text{pred}_t([0..i+1), i+1); \text{pred}_f([0..i+1), i+1))\}$ 
  6. }
 $\{(\exists i. i = n; \text{pred}_t([0..i)); \text{pred}_f([0..i]))\} \rightsquigarrow \{(\exists i. \text{true}; \text{pred}_t([0..n)); \text{pred}_f([0..n]))\}$ 

```

Fig. 12. Outlining the reasoning for the loop in Fig. 5

line 3, Algorithm 1 generates an invariant where i serves as the iterator and “[0.. i]” denotes the bag representation. The verification then establishes that this invariant holds through each loop iteration – specifically, the program states after lines 3 and 5 are isomorphic with respect to the iterator’s value. The final program state is derived by conjoining with the negated loop guard, i.e., ($i=n$).

This process employs heuristics and is inherently incomplete: it may fail to capture the complete bag range or the precise iterator (which can be improved by incorporating the loop iterator’s initial value). Nevertheless, the verification remains sound, as [FV-While] only succeeds when valid invariants are provided.

4.2 Trace Inclusion

As shown in Fig. 13, we use $(\mathcal{H} \vdash \theta_1 \sqsubseteq_\pi \theta_2)$ to denote the inclusion between two traces, where $\mathcal{H}$ contains a set of proof hypotheses, and when omitted it is initialized with $\{\}$. [Inc-Disprove] disproves the inclusions when the antecedent is nullable (containing the empty trace ϵ), while the consequent is not. [Inc-Reoccur] proves an inclusion by leveraging existing hypotheses in the proof context $\mathcal{H}$ that soundly justify the current goal. [Inc-Unfold] serves as the inductive step, unfolding inclusions – proof of the original inclusion succeeds if all derivative inclusions succeed. Termination of the rewriting is guaranteed because the set of derivatives to be considered is finite, and possible cycles are detected via memorization ([Inc-Reoccur]). We define the soundness of these rules in Theorem 1.

$$\begin{array}{c}
\text{[Inc-Disprove]} \\
\frac{\delta(\theta_1) \wedge \neg \delta(\theta_2)}{\mathcal{H} \vdash \theta_1 \not\sqsubseteq_\pi \theta_2}
\end{array}
\quad
\begin{array}{c}
\text{[Inc-Reoccur]} \\
\frac{(\theta_1 \sqsubseteq_\pi \theta_2) \in \mathcal{H}}{\mathcal{H} \vdash \theta_1 \sqsubseteq_\pi \theta_2}
\end{array}
\quad
\begin{array}{c}
\text{[Inc-Unfold]} \quad \forall A \in \text{fst}(\theta_1). \\
\frac{(\theta_1 \sqsubseteq_\pi \theta_2) ++ \mathcal{H} \vdash \mathcal{D}_A^\pi(\theta_1) \sqsubseteq_\pi \mathcal{D}_A^\pi(\theta_2)}{\mathcal{H} \vdash \theta_1 \sqsubseteq_\pi \theta_2}
\end{array}$$

Fig. 13. Trace inclusion checking rules

To facilitate these inclusion rules, we provide the definitions of the deployed auxiliary functions: *Nullable* (δ) at Definition 1, *First* (fst) at Definition 3, and *Derivative* ($\mathcal{D}_A^\pi(\theta)$) at Definition 4. Informally, the First function $\text{fst}(\theta)$ computes a set of possible initial events from θ . The Derivative function $\mathcal{D}_A^\pi(\theta)$ eliminates an event A from the head of θ and returns what remains.

Definition 3 (First). Let $\text{fst}(\theta)$ be the set of initial events derivable from θ .

$$\begin{aligned}
\text{fst}(\perp) &= \text{fst}(\epsilon) = \{\} & \text{fst}(A) &= \{A\} & \text{fst}(\theta_1 \vee \theta_2) &= \text{fst}(\theta_1) \cup \text{fst}(\theta_2) & \text{fst}(\theta^*) &= \text{fst}(\theta) \\
\text{fst}(\theta_1 \wedge \theta_2) &= \text{fst}(\theta_1) \cap \text{fst}(\theta_2) & \text{fst}(\theta_1 \cdot \theta_2) &= \begin{cases} \text{fst}(\theta_1) \cup \text{fst}(\theta_2) & \text{if } \delta(\theta_1) = \text{true} \\ \text{fst}(\theta_1) & \text{if } \delta(\theta_1) = \text{false} \end{cases}
\end{aligned}$$

Definition 4 (Derivative). The partial derivative $\mathcal{D}_A^\pi(\theta)$ eliminates an event A from the head of a trace θ , defined as follows:

$$\begin{aligned}
\mathcal{D}_{\mathbf{A}}^{\pi}(\perp) &= \mathcal{D}_{\mathbf{A}}^{\pi}(\epsilon) = \perp & \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1 \wedge \theta_2) &= \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1) \wedge \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_2) & \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1 \vee \theta_2) &= \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1) \vee \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_2) \\
\mathcal{D}_{\mathbf{A}}^{\pi}(\theta^{\star}) &= \mathcal{D}_{\mathbf{A}}^{\pi}(\theta) \cdot \theta^{\star} & \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1 \cdot \theta_2) &= \begin{cases} (\mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1) \cdot \theta_2) \vee \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_2) & \text{if } \delta(\theta_1) = \text{true} \\ \mathcal{D}_{\mathbf{A}}^{\pi}(\theta_1) \cdot \theta_2 & \text{if } \delta(\theta_1) = \text{false} \end{cases}
\end{aligned}$$

Definition 5 serves as the base case for Definition 4, computing the derivatives between two events. While these auxiliary functions were originally designed to solve inequalities between regular expressions, they only supported cases where events were simple alphabets [4], with derivatives computed via lexical comparison. In our work, events could be trace predicates or parameterized with program variables. To handle such cases, we thus extend the Derivative function with pure constraints. In Definition 5, the definition of event derivatives adheres strictly to the semantic model (cf. Fig. 10). Notice that $\mathbf{A}$ takes one of two forms: a positive event “ $ev(t)$ ”, or a trace predicate, as they are the only cases that are derivable from the program executions. When both $\mathbf{A}$ and $\mathbf{B}$ are trace predicates, they must first be normalized into the same bag “ B ” and then processed using the trace subtraction operator (detailed in Sec. 4.3).

Definition 5 (Derivative for Events). *Given any two events $\mathbf{A}$, $\mathbf{B}$, the derivative of $\mathbf{B}$ with respect to $\mathbf{A}$, i.e., $\mathcal{D}_{\mathbf{A}}^{\pi}(\mathbf{B})$ is defined as follows: ($\perp$ for the unmentioned scenarios)*

$$\begin{aligned}
\mathcal{D}_{ev(t)}^{\pi}(ev(t')) &= \begin{cases} \epsilon & \text{if } \pi \Rightarrow (t=t') \\ \perp & \text{otherwise} \end{cases} & \mathcal{D}_{ev(t)}^{\pi}(\neg_{-}(t')) &= \begin{cases} \epsilon & \text{if } \pi \not\Rightarrow (t=t') \\ \perp & \text{otherwise} \end{cases} \\
\mathcal{D}_{ev(t)}^{\pi}(\neg ev'(t')) &= \begin{cases} \perp & \text{if } ev = ev' \text{ and } \pi \leq (t=t') \\ \epsilon & \text{otherwise} \end{cases} \\
\mathcal{D}_{ev(t)}^{\pi}(_) &= \epsilon & \mathcal{D}_{ev(t)}^{\pi'}(\Lambda_i^B(\dots)) &= \Lambda_i^B(\dots) \\
\mathcal{D}_{\Lambda_i^B(\bigvee(\pi_2 \wedge \theta_2))}^{\pi'}(\Lambda_i^B(\bigvee(\pi_1 \wedge \theta_1))) &= \Lambda_i^B(\bigvee(\pi_1 \wedge \pi_2 \wedge \theta')) & \text{where } \theta_1 \ominus \theta_2 &\hookrightarrow_{\pi'} \theta'
\end{aligned}$$

Theorem 1 (Soundness of Trace Inclusion). *For all π , θ_1 , θ_2 , s and ρ , given $\theta_1 \sqsubseteq_{\pi} \theta_2$, it means that if $s, \rho \models \pi \wedge \theta_1$, then $s, \rho \models \pi \wedge \theta_2$.*

Proof. By induction on the derivation of $\theta_1 \sqsubseteq_{\pi} \theta_2$.

4.3 Subtracting Traces from Future Conditions

As shown in Fig. 14, we use $F \ominus \theta \hookrightarrow_{\pi} F'$ to denote the subtraction of the trace θ from the given future condition F , resulting in a “left-over” future condition F' , which we call a *residue*. Rule [TS-Base] captures the base case where the trace to subtract is an empty trace, leaving the future condition unchanged. Rule [TS-Bot] handles the case where the subtracted trace is $\perp$, leading to false future conditions. Rule [TS-Ind] disjunctively unions all the subtraction results

$$\begin{array}{c}
\frac{}{F \odot \epsilon \hookrightarrow_{\pi} \theta} \text{[TS-Base]} \quad \frac{}{F \odot \perp \hookrightarrow_{\pi} \perp} \text{[TS-Bot]} \quad \frac{\forall A \in \text{fst}(\theta_2). \quad \mathcal{D}_{\underline{A}}^{\pi}(\theta_1) \odot \mathcal{D}_{\underline{A}}^{\pi}(\theta_2) \hookrightarrow_{\pi} \theta_i}{\theta_1 \odot \theta_2 \hookrightarrow_{\pi} \bigvee \theta_i} \text{[TS-Ind]} \\
\\
\frac{\frac{F_3 \odot \theta \hookrightarrow_{\pi} F_4}{F_3 \sqsubseteq_{\pi} F_1} \quad F_2 \sqsubseteq_{\pi} F_4}{F_1 \odot \theta \hookrightarrow_{\pi} F_2} \text{[TS-Conseq]} \quad \frac{F_1 \odot \theta \hookrightarrow_{\pi} F_2 \quad \theta' \sqsubseteq_{\pi} \theta}{F_1 \odot \theta' \hookrightarrow_{\pi} F_2} \text{[TS-Trans]}
\end{array}$$

Fig. 14. Trace subtraction rules

from the derivatives. Rule [TS-Conseq] allows for strengthening the subtraction residues and weakening the input future conditions. Rule [TS-Trans] captures the case where we can strengthen the subtracted trace. We define and prove the soundness of these rules in Theorem 2, which indicates that in a sound subtraction, the residue can only be strengthened.

Theorem 2 (Soundness of Trace Subtraction). *For all π, F, F', θ, s and ρ , given $F \odot \theta \hookrightarrow_{\pi} F'$ it means that if $s, \rho \models \pi \wedge (\theta \cdot F')$ then $s, \rho \models \pi \wedge F$.*

Proof. By induction on the derivation of $F \odot \theta \hookrightarrow_{\pi} F'$.

4.4 Soundness of Forward Reasoning

We here define the soundness of the forward rules and highlight the key lemmas. Theorem 3 presents the soundness for the generalized forward rules, where rules like $\{\pi\} e \{\Delta\}$ are generalized into the form of $\{(\pi; \epsilon; _*)\} e \{\Delta\}$. The soundness states that for any given expression e , starting from a concrete model which satisfies the pre-state, when e evaluates to a value v , the resulting concrete model satisfies one of the concluded post-summaries, which guarantees that all the forward rules soundly over-approximating e 's behavior and strengthening the final future conditions.

Theorem 3 (Soundness of the Generalised Forward Rules). *For all $e, \pi, \theta_1, F, F', s_1, s_2, \rho_1, \rho_2, v$, given $\{(\pi; \theta_1; F)\} e \{\Delta\}$, $[s_1, \rho_1, F, e] \longrightarrow [s_2, \rho_2, F', v]$, and $s_1, \rho_1 \models \pi \wedge \theta_1$, there exists $(\pi'; \theta_2; F'') \in \Delta$ such that $(s_2 \text{ ++ res} = v), \rho_2 \models \pi' \wedge \theta_2$ and $F'' \sqsubseteq_{\pi'} F'$.*

Proof. By induction on the structure of e , and applying Lemma 1 and Lemma 2 when proving the sequencing rule [FV-Seq] and the structural rule [FV-Struct].

Lemma 1 (Strengthening the Future Conditions from the Instrumented Semantics). *For all, $s_1, s_2, \rho_1, \rho_2, F_1, F_2, F_3, v$, given $[s_1, \rho_1, F_1, e] \longrightarrow [s_2, \rho_2, F_2, v]$ and $F_3 \sqsubseteq_{\text{Pure}(s_1)} F_1$, there exists F_4 , such that, $[s_1, \rho_1, F_3, e] \longrightarrow [s_2, \rho_2, F_4, v]$ and $F_4 \sqsubseteq_{\text{Pure}(s_2)} F_2$.*

Proof. By induction on $[s_1, \rho_1, F_1, e] \longrightarrow [s_2, \rho_2, F_2, v]$.

Lemma 2 (Approximating the Concrete Trace Subtraction). *For all $F_1, \theta, F_2, s, \rho$, given $F_1 \odot \theta \hookrightarrow_{\pi} F_2$ and $s, \rho \models \pi \wedge \theta$, then exists F_3 such that $F_1 \odot_{\text{lin}} \rho \hookrightarrow_{\text{Pure}(s)} F_3$ and $F_2 \sqsubseteq_{\text{Pure}(s)} F_3$.*

Proof. By induction on $F_1 \ominus \theta \hookrightarrow_\pi F_2$.

Lemma 1 states that in the instrumented semantics, starting with a stronger future condition leads to stronger resulting future conditions. In Lemma 2, we use $\ominus_{lin}$ to denote subtracting a linear sequence of events ρ from a given future condition without any approximation. It states that the trace subtraction $\ominus$ introduces approximation and results in stronger residues. Additionally, Table 1 presents a set of propositions essential for improving completeness. For example, the trace subtraction for the predicate in Fig. 7 is performed after applying propositions (10) and (12). The detailed derivation are shown in Appendix A.

Table 1. Selected propositions for reasoning future conditions

$\theta_1 \wedge \theta_2 \leftrightarrow \theta_2 \wedge \theta_1$	(1)	$\theta_1 \wedge (\theta_2 \wedge \theta_3) \leftrightarrow (\theta_1 \wedge \theta_2) \wedge \theta_3$	(2)
$\perp \cdot \theta = \theta \cdot \perp \rightarrow \perp$	(3)	$(\theta_1 \cdot \theta_2 \neq \perp) \rightarrow (\theta_1 \neq \perp) \wedge (\theta_2 \neq \perp)$	(4)
$_{-}^* \sqsubseteq_\pi \theta \rightarrow (\theta =_{-}^*)$	(5)	$(\theta_1 \wedge \theta_2 \neq \perp) \rightarrow (\theta_1 \neq \perp) \wedge (\theta_2 \neq \perp)$	(6)
$(_{-}^* \ominus \mathbf{A} \hookrightarrow_\pi \theta) \rightarrow (\theta =_{-}^*)$	(7)	$\theta_1 \sqsubseteq_\pi \theta_2 \rightarrow (\theta_2 \sqsubseteq_\pi \theta_3 \rightarrow \theta_1 \sqsubseteq_\pi \theta_3)$	(8)
$(_{-}^* \ominus_{lin} \mathbf{A} \hookrightarrow_\pi \theta) \rightarrow (\theta =_{-}^*)$	(9)	$\Lambda_i^{B_1 \cup B_2}(\dots) \leftrightarrow \Lambda_i^{B_1}(\dots) \wedge \Lambda_i^{B_2}(\dots)$	(10)
$\Lambda_i^B(\bigvee(\pi \wedge_{-}^*)) \rightarrow_{-}^*$	(11)	$\Lambda_i^{\{v\}}(\bigvee(\pi \wedge \theta)) \rightarrow (\bigvee(\pi \wedge \theta))[v/i]$	(12)

5 Evaluation and Case Studies

We formalize the soundness proofs presented in this work using Coq, publicly available from our artifact [2]. The Coq development comprises approximately 2,300 lines of code (LoC). We also prototype our system into an automated verification tool, implemented in 3,600 LoC of OCaml, demonstrating its applicability in verifying real-world programs. Experiments were done on a MacBook

Table 2. Selected APIs with their specifications in different usage contexts

Category	Example APIs	Future Conditions
1. File Ops	fopen, open	Finally to close the file descriptor
	fclose, close	Globally do not access the file descriptor
		Read-only files cannot be written to
2. Threads	pthread_create	Finally to pthread_join or detach the thread
	pthread_mutex_lock	Finally to pthread_mutex_unlock
3. Memory	free	Globally do not access the pointer
	malloc	Finally free the new pointer
	realloc	Globally the old pointer is not accessed & finally free the new pointer
4. Sockets	socket	Finally to close the socket
5. Database	sqlite3_open	Finally to sqlite3_close the connection
6. URV/NPD	fgets, gethostbyaddr	Check the return value immediately after calls

with a 2.6 GHz 6-Core Intel i7 processor. The source code and the evaluation benchmark are openly accessible [2].

Experiment Setup. Table 2 categorizes APIs by functionality and specifies the future conditions required for their safe usage. The table organizes these APIs into six categories (1-6), covering operations such as file I/O, thread synchronization, memory management, and more. These conditions were manually derived from official documentation and common usage patterns. The complete set of formalized specifications are provided in Appendix B. In particular, we treat the *unchecked return value* (URV) and *null-pointer dereference* (NPD) vulnerabilities to be a critical use case for future conditions. URVs and NPDs occur when a program fails to validate the return value of a function, potentially leading to crashes or undefined behavior. To mitigate such risks, developers must ensure that all function return values are checked and handled appropriately.

Table 3. Experimental Results

Category	LoC	PrimS	InferredS	InferredInv	Report/Exp.	Time(s)
1	656	8	29	7	14/12	10.05
2	330	4	25	1	4/4	1.97
3	424	6	30	11	25/23	8.87
4	103	2	6	1	3/3	1.76
5	108	4	6	0	4/4	1.97
6	67	10	5	0	5/5	0.56
Total	1,688	34	101	20	55/51	25.18

Experimental Results. Our evaluation uses real-world C programs sourced from: (i) the CWE database (containing diverse vulnerability types); (ii) API usage tutorials (demonstrating correct practices); and (iii) GitHub repositories (featuring real-world usage of critical APIs). The benchmark contains 51 manually verified API misuse violations, serving as ground truth for evaluation. Results are summarized in Table 3, with the following metrics: **PrimS** is the number of primitive specifications (manually provided, avg. 4.5 LoC/spec, cf. Appendix B), **InferredS** is the number of inferred specifications (equals to the number of analyzed function declarations), **InferredInv** is the number of inferred trace invariants, **Report/Exp.** stands for reported violations/ground truth violations, and **Time** records the total verification time in seconds. Our evaluation spans 1,688 LoC across six distinct categories, with verification and inference applied to 101 functions. The tool reports 55 violations, including all 51 manually verified true positives and 4 false positives (incorrectly flagged safe code). Total verification time is 25.18 seconds, with processing time scaling proportionally to the program length and the number of primitive specifications.

Expressiveness Limitations. The observed false positives arise from three limitations: (i) trace invariant inference fails under non-structured control flow

```

1 void false_positive1() {
2   int** ptr1= malloc(4);
3   int*  ptr2= malloc(4);
4   *ptr1 = ptr2;
5   free(*ptr1);
6   free(ptr1); }
7 False positive: Memory Leak!

```

Fig. 15. False positive example for (ii)

```

1 void false_positive2
2   (const char* filename) {
3   int r = unlink(filename);
4   if (r == 0) {
5     int fd=fopen(filename,"r");
6     assert(fd == NULL);}}
7 False positive: Unclosed File!

```

Fig. 16. False positive example for (iii)

(e.g., goto statements, infinite loops); (ii) incomplete modeling for memory usage; and (iii) restriction to first-order constraints. We next illustrate (ii) and (iii) with concrete examples. As shown in Fig. 15, the pointer `ptr1` points to the address of `ptr2` (assigned at line 4). In this code, freeing `*ptr1` (the content of `ptr1`) and then `ptr1` itself is safe and correctly avoids memory leaks. However, the deployed pure arithmetic logic fails to recognize the *points-to* relationship established at line 4. As a result, it incorrectly reports a memory leak, suggesting that `ptr2` is never freed – even though it is freed via `*ptr1`. This can be mitigated by extending the basic logic with *points-to* relations. As demonstrated in Fig. 16, the `unlink` function removes a file from the filesystem when it returns 0. Consequently, the subsequent call to `fopen` at line 5 will always fail (returning `NULL`), making the code correct – the assertion at line 6 will always succeed. However, because our tool does not model `unlink`'s side effects – specifically, that future `fopen` calls on the same filename must fail – it incorrectly reports a file descriptor leak. Tracking external system for file states would resolve this limitation. Despite these limitations, our framework remains the first sound verification system capable of detecting a broad range of violations via temporal logic encoding – providing a foundation for these aforementioned extensions.

Case Study: Inter-procedural Analysis & Conditional Violation. Fig. 17 presents a non-trivial double-free example, drawn from prior works [9, 15]. The *foo* function takes a pointer p and returns either a newly allocated pointer or the existing pointer $p \rightarrow f$, based on the value of $p \rightarrow \text{flag}$. In *main* function, a local *st* structure is created, memory is allocated for $p.f$, and *foo* is called. The potential double-free occurs because: if $p.\text{flag}$ is false, q and $p.f$ point to the same memory (line 4). Both q and $p.f$ are then freed (lines 10, 11), causing a double-free when $p.\text{flag}$ was false (line 11). This example is non-trivial because it requires a precise inter-procedural analysis and path sensitivity.

The verification proceeds as follows: Lines 2 and 7 initialize the program states using existentially quantified variables. Lines 3 and 4 reason about the two branches of the conditional and creates the existential variable l for the newly allocated heap address. By integrating the results, line 5 leads to a disjunctive post-summary which says that when $p \rightarrow \text{flag}$ is true, the program returns a newly

```

1. int *foo(struct st *p){
2.   int *q;
   { $\exists q. true; \epsilon; \_*$ }
3.   if (p->flag) q = malloc(1); [FV-Call][FV-Assign][FV-Cond][FV-Struct]
   { $\exists q, l. (p \rightarrow flag) \wedge q = l; \text{malloc}(l); \mathcal{F}(free(l))$ }
4.   else q = p->f; [FV-Assign][FV-Cond][FV-Struct]
   { $\exists q. \neg(p \rightarrow flag) \wedge q = (p \rightarrow f); \epsilon; \_*$ }
5.   return q; } [FV-Value][FV-Cond][FV-Struct][FV-Seq]
   { $\exists l. (p \rightarrow flag) \wedge res = l; \text{malloc}(l); \mathcal{F}(free(l)) \vee \neg(p \rightarrow flag) \wedge res = (p \rightarrow f); \epsilon; \_*$ }

6. int main(){
7.   struct st p; int *q;
   { $\exists p, q. true; \epsilon; \_*$ }
8.   p.f = malloc(1); [FV-Call][FV-Assign][FV-Struct]
   { $\exists p, q, l. p.f = l; \text{malloc}(l); \mathcal{F}(free(l))$ }
9.   q = foo(&p); [FV-Call][FV-Assign][FV-Struct][FV-Seq]
   { $\exists p, q, l, l_1. (p.flag) \wedge p.f = l \wedge q = l_1; \text{malloc}(l) \cdot \text{malloc}(l_1); \mathcal{F}(free(l)) \wedge \mathcal{F}(free(l_1))$ }
   { $\vee \exists p, q, l. \neg(p.flag) \wedge q = p.f \wedge p.f = l; \text{malloc}(l); \mathcal{F}(free(l))$ }
10.  free(q); [FV-Call][FV-Struct][FV-Seq]
   { $\exists p, q, l, l_1. (p.flag) \wedge p.f = l \wedge q = l_1; \text{malloc}(l) \cdot \text{malloc}(l_1) \cdot \text{free}(l_1); \mathcal{F}(free(l)) \wedge \mathcal{G}(!\_ (l_1))$ }
   { $\vee \exists p, q, l. \neg(p.flag) \wedge q = p.f \wedge p.f = l; \text{malloc}(l) \cdot \text{free}(l); \mathcal{G}(!\_ (l))$ }
11.  free(p.f); } [FV-Call][FV-Struct][FV-Seq]
   { $\exists p, q, l, l_1. (p.flag) \wedge p.f = l; \text{malloc}(l) \cdot \text{malloc}(l_1) \cdot \text{free}(l_1) \cdot \text{free}(l); \mathcal{G}(!\_ (l)) \wedge \mathcal{G}(!\_ (l_1))$ }
   { $\vee \exists p, q, l. \neg(p.flag) \wedge q = p.f \wedge p.f = l; \text{malloc}(l) \cdot \text{free}(l) \cdot \text{free}(l); \perp$  ← X}

```

Fig. 17. Inter-procedural analysis for detecting a conditional double free violation

allocated heap location l , and there is a future condition to finally free l ; when $p \rightarrow flag$ is false, it returns $p \rightarrow f$ and have no meaningful future conditions.

When calling the function *foo*, line 9 retrieves its specification (obtained after line 5), renames all its existential variables using fresh variables, and composes the instantiated specification into the current summaries. In line 10, under the condition $\neg(p.flag)$, freeing q updates the propagated future conditions from $\mathcal{F}(free(l))$ to $\mathcal{G}(!_ (l))$. Subsequently, in line 11, when freeing $p.f$, it detects the double free under the pure constraint $\pi = (\exists p, q, l. \neg(p.flag) \wedge q = p.f \wedge p.f = l)$ – the following trace subtraction step results in *false*: $\mathcal{G}(!_ (l)) \ominus free(p.f) \hookrightarrow_{\pi} \perp$.

6 Related Work

Statically Checked Contracts. Typestate systems [16], recently revisited in various forms [7, 13, 18, 3], may also be used to a similar outcome as future conditions. These systems associate an object’s state with its type, enforcing valid operation sequences based on that state, which is useful in object-oriented programming. Both typestates and future conditions model the correct execution

order of operations on entities. However, *typestates* operate on objects, while *future conditions* operate on program variables. Additionally, implementing *typestate* systems often requires extensive boilerplate code. For instance, developers must meticulously define state transitions in a global view and manage associated types, increasing complexity in code maintenance and readability. In contrast, *future conditions* offer a more concise and modular way to specify and verify temporal properties without requiring explicit global state transitions. Their modularity stems from the fact that each *future condition* encapsulates temporal constraints specific to its associated function, independent of any global state.

Rust’s type system [1] primarily enforces memory safety (ownership, borrowing, lifetimes), while the *typestate pattern* extends this by encoding state transitions into the type system, ensuring operations are only valid in certain states (e.g., a `File<Open>` can be read, but a `File<Closed>` cannot). However, *typestates* do not explicitly handle temporal properties (e.g., “eventually, this operation must happen”). Such properties must instead be verified separately against the global state transitions. Our approach unifies and extends both *typestate* systems and Rust’s type system by: encoding memory safety using LTL formulas and enabling temporal reasoning (e.g., “this resource remains valid until condition π holds”) alongside memory safety. Lastly, effect systems are also closely related, as they constrain which effects can be performed. While ordinary effect systems do not consider the order of effects, sequential effect systems [17, 8, 14] do. However, they follow the form of pre-/post-conditions, which is inadequate for describing future conditions in a modular style.

Dynamically Checked Contracts. Trace contracts [12] are for specifying and verifying properties of sequences of function calls and returns in the Racket programming language. They allow developers to define predicates over the sequence of values that flow through function calls, enabling the detection of violations of expected behaviors across multiple function invocations. By monitoring programs at run time, trace contracts are able to take advantage of the precision that run-time checking offers, which possibly goes beyond statically decidable properties. Similar to *typestate* systems, trace contracts describes a global view of the protocols, while *future conditions* provide a more modular and local view of each effectful operation. Moreover, apart from the run-time overhead and increased resource consumption, trace contracts has limited expressiveness for imperative assignments and effectful loops, which are now supported in our verification framework.

Bug Finding at Scale. While Pulse [10] (based entirely on separation logic) and ProveNFix [15] (grounded in temporal logic) both use specification inference to detect memory and resource violations, our work generalizes both solution and yields a more comprehensive solution. Unlike Pulse, we eliminate the need for implicit side-checks, and compared to ProveNFix, we avoid repeated analysis while maintaining full temporal violation detection capabilities.

7 Conclusion

Future conditions extend traditional pre- and post-conditions by specifying temporal behaviors and states that emerge after a function call completes. This work tackles the central question: “*How can we efficiently and soundly reason about future conditions using temporal logic?*” We propose a compositional verification framework that propagates future conditions independently of contextual code. Our solution introduces novel trace inclusion and trace subtraction mechanisms to facilitate a more efficient propagation of future conditions, where they are processed once per method declaration. Additionally, we present an approach for verifying effectful loops using trace and future condition predicates. The soundness of our system is formally proven, and its practicality is demonstrated through experimental results and non-trivial case studies.

8 Acknowledgments

This research is supported by the Ministry of Education, Singapore, under its MOE Academic Research Fund Tier 3 (RIE2025) (MOE Award No: MOET-MOET32021-0001), and by the Ministry of Education, Singapore, under the Academic Research Fund Tier 1 (FY2023). We thank anonymous reviewers for their insightful comments, which led to improvements in the paper presentation.

Data Availability

The source code of the tool, the dataset, proofs, are available from [2].

References

1. The Rust Programming Language. <http://rust-lang.org>, 2017.
2. Benchmark and Source Code. <https://zenodo.org/records/15371095>, 2025.
3. J. Aldrich, J. Sunshine, D. Saini, and Z. Sparks. Typestate-oriented programming. In S. Arora and G. T. Leavens, editors, *Companion to the 24th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2009, October 25-29, 2009, Orlando, Florida, USA*, pages 1015–1022. ACM, 2009.
4. V. Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996.
5. CWE. Cwe-416: Use after free. <https://cwe.mitre.org/data/definitions/416.html>.
6. L. M. de Moura and N. S. Bjørner. Z3: an efficient SMT solver. In C. R. Ramakrishnan and J. Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.
7. C. Jaspan and J. Aldrich. Checking framework interactions with relationships. In S. Drossopoulou, editor, *ECOOP 2009 - Object-Oriented Programming, 23rd European Conference, Genoa, Italy, July 6-10, 2009. Proceedings*, volume 5653 of *Lecture Notes in Computer Science*, pages 27–51. Springer, 2009.

8. E. Koskinen and T. Terauchi. Local temporal reasoning. In T. A. Henzinger and D. Miller, editors, *Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS), CSL-LICS '14, Vienna, Austria, July 14 - 18, 2014*, pages 59:1–59:10. ACM, 2014.
9. J. Lee, S. Hong, and H. Oh. Memfix: static analysis-based repair of memory deallocation errors for C. In G. T. Leavens, A. Garcia, and C. S. Pasareanu, editors, *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04-09, 2018*, pages 95–106. ACM, 2018.
10. Meta. Infer static analyzer. <https://github.com/facebook/infer>, 2025.
11. B. Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992.
12. C. Moy and M. Felleisen. Trace contracts. *J. Funct. Program.*, 33, 2023.
13. R. Pucella and J. A. Tov. Haskell session types with (almost) no class. In A. Gill, editor, *Proceedings of the 1st ACM SIGPLAN Symposium on Haskell, Haskell 2008, Victoria, BC, Canada, 25 September 2008*, pages 25–36. ACM, 2008.
14. Y. Song, D. Foo, and W. Chin. Automated temporal verification for algebraic effects. In I. Sergey, editor, *Programming Languages and Systems - 20th Asian Symposium, APLAS 2022, Auckland, New Zealand, December 5, 2022, Proceedings*, volume 13658 of *Lecture Notes in Computer Science*, pages 88–109. Springer, 2022.
15. Y. Song, X. Gao, W. Li, W. Chin, and A. Roychoudhury. ProveNFix: Temporal property-guided program repair. *Proc. ACM Softw. Eng.*, 1(FSE):226–248, 2024.
16. R. E. Strom and S. Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Trans. Software Eng.*, 12(1):157–171, 1986.
17. R. Tate. The sequential semantics of producer effect systems. In R. Giacobazzi and R. Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, pages 15–26. ACM, 2013.
18. R. Wolff, R. Garcia, É. Tanter, and J. Aldrich. Gradual typestate. In M. Mezini, editor, *ECCOP 2011 - Object-Oriented Programming - 25th European Conference, Lancaster, UK, July 25-29, 2011 Proceedings*, volume 6813 of *Lecture Notes in Computer Science*, pages 459–483. Springer, 2011.

A Trace Subtraction after Normalization

$$\begin{aligned}
 & \text{pred}_f([0..5], \text{arr}) \ominus \text{free}(\text{arr}[0]) && (\text{Table 1-10}) \\
 \hookrightarrow_{\pi} & (\text{pred}_f(0, \text{arr}) \wedge \text{pred}_f([1..5], \text{arr})) \ominus \text{free}(\text{arr}[0]) && (\text{Table 1-11}) \\
 \hookrightarrow_{\pi} & (\mathcal{F}(\text{free}(\text{arr}[0])) \ominus \text{free}(\text{arr}[0])) \wedge \text{pred}_f([1..5], \text{arr}) && (\text{Definition 4}) \\
 \hookrightarrow_{\pi} & _ * \wedge \text{pred}_f([1..5], \text{arr})
 \end{aligned}$$

B Annotated Specifications

```

1 //----- 1 File -----
2 /*@ open(path, flag) =

```

```

3 REQ TRUE
4 ENS (ex fd: fd=-1 ; emp ; (!_(fd))^* ; fd)
5 \/(ex fd : !(fd=-1) /\ flag=0 ; open(fd) ; (!write(fd))
   ^* /\ (!close(fd))^* . close(fd) . (_)^* ; fd)
6 \/(ex fd : !(fd=-1) /\ flag=1 ; open(fd) ; (!read(fd))^*
   /\ (!close(fd))^* . close(fd) . (_)^* ; fd)
7 \/(ex fd : !(fd=-1) /\ flag=2 ; open(fd) ; (!close(fd))
   ^* . close(fd) . (_)^* ; fd) @*/
8
9 /*@ fopen(path, flag) =
10 REQ TRUE
11 ENS (ex fd: fd=-1 ; emp ; (!_(fd))^* ; fd)
12 \/(ex fd : !(fd=-1) /\ flag=0 ; open(fd) ; (!write(fd))
   ^* /\ (!close(fd))^* . close(fd) . (_)^* ; fd)
13 \/(ex fd : !(fd=-1) /\ flag=1 ; open(fd) ; (!read(fd))^*
   /\ (!close(fd))^* . close(fd) . (_)^* ; fd)
14 \/(ex fd : !(fd=-1) /\ flag=2 ; open(fd) ; (!close(fd))
   ^* . close(fd) . (_)^* ; fd) @*/
15
16 /*@ close(fd) =
17 REQ TRUE
18 ENS (ex r : r=unit ; close(fd) ; (_)^* ; r) @*/
19 /*@ fclose(fd) =
20 REQ TRUE
21 ENS (ex r : r=unit ; close(fd) ; (_)^* ; r) @*/
22 /*@ write(fd, buf, size) =
23 REQ TRUE
24 ENS (ex r : TRUE ; write(fd) ; (_)^* ; r) @*/
25 /*@ read(fd, buf, size) =
26 REQ TRUE
27 ENS (ex r : TRUE ; read(fd) ; (_)^* ; r) @*/
28 /*@ exit(code) =
29 REQ TRUE
30 ENS (: TRUE ; exit() ; (_)^* ; code; -2 ) @*/
31 /*@ return(t) =
32 REQ TRUE
33 ENS (: TRUE ; emp ; (_)^* ; t; -1) @*/
34
35 //----- 2 Thread -----
36 /*@ pthread_create(a, b, c, d) =
37 REQ TRUE
38 ENS (ex l : TRUE ; pthread_create(a) ; (!
   pthread_join(a))^* . pthread_join(a) . (_)^* ;
   l) @*/
39 /*@ pthread_join(a, b) =
40 REQ TRUE
41 ENS (ex r : TRUE ; pthread_join(a) ; (_)^* ; r) @*/
42 /*@ pthread_mutex_lock(a) =
43 REQ TRUE

```

```

44     ENS (ex l : TRUE ; pthread_mutex_lock(a) ; (!
        pthread_mutex_unlock(a))^* . pthread_mutex_unlock
        (a) .  (_)^* ; l) @*/
45 /*@ pthread_mutex_unlock(a) =
46     REQ TRUE
47     ENS (ex r : TRUE ; pthread_mutex_unlock(a) ; (_)^* ; r
        ) @*/

48
49 //----- 3 Memory -----
50 /*@ free(ptr) =
51     REQ TRUE
52     ENS (ex r : r=unit ; free(ptr) ; (!_(ptr))^* ; r) @*/
53 /*@ malloc(size) =
54     REQ TRUE
55     ENS (ex l : TRUE ; malloc(l) ; (!free(l))^* . free
        (l) .  (_)^* ; l) @*/
56 /*@ realloc(ptr, size) =
57     REQ TRUE
58     ENS (ex l : TRUE ; realloc(l) ; (!_(ptr))^* /\ (!
        free(l))^* . free(l) .  (_)^* ; l) @*/
59 /*@ return(t) =
60     REQ TRUE
61     ENS (: TRUE ; emp ; (_)^* ; t; -1) @*/
62 /*@ printf(t, p) =
63     REQ TRUE
64     ENS (: TRUE ; printf(p) ; (_)^* ; unit) @*/
65 /*@ strncpy(a, b, c) =
66     REQ TRUE
67     ENS (: TRUE ; strncpy(a) ; (_)^* ; unit) @*/
68
69 //----- 4 Socket -----
70 /*@ socket(a, b, c) =
71     REQ TRUE
72     ENS (ex l : l>=0 ; socket() ; (!close(l))^* .
        close(l) .  (_)^* ; l) @*/
73 /*@ close(fd) =
74     REQ TRUE
75     ENS (ex r : r=unit ; close(fd) ; (_)^* ; r) @*/
76
77 //----- 5 Database -----
78 /*@ sqlite3_open(a, b) =
79     REQ TRUE
80     ENS (ex l : TRUE ; sqlite3_open(b) ; (!sqlite3_close
        (b))^* . sqlite3_close(b) .  (_)^* ; l) @*/
81 /*@ sqlite3_close(a) =
82     REQ TRUE
83     ENS (ex r : TRUE ; sqlite3_close(a) ; (_)^* ; r) @*/
84
85 //----- 6 Protocol -----
86 /*@ fgets(a, b, c) =

```



```

87     REQ  TRUE
88     ENS (ex r : r=NULL ; fgets(a) ; return() ; r) @*/
89 /*@ returnChunkSize(a) =
90     REQ  TRUE
91     ENS (ex r : r=-1 ; returnChunkSize(a) ; return() ; r)
        @*/
92 /*@ malloc(a) =
93     REQ  TRUE
94     ENS (ex r : r=NULL ; malloc(a) ; return() ; r) @*/
95 /*@ gethostbyaddr(a, b, c) =
96     REQ  TRUE
97     ENS (ex r : r=NULL ; gethostbyaddr(a) ; (!_(r))^* ; r
        ) @*/
98 /*@ pthread_mutex_lock(a) =
99     REQ  TRUE
100     ENS (ex r : !(r=0) ; pthread_mutex_lock(a) ; (!_(a))
        ^* ; r) @*/
101 /*@ pthread_mutex_unlock(a) =
102     REQ  TRUE
103     ENS (ex r : TRUE ; pthread_mutex_unlock(a) ; ( _ )^* ;
        r) @*/
104 /*@ strncpy(a, b, c) =
105     REQ  TRUE
106     ENS (ex r : TRUE ; strncpy(b) ; ( _ )^* ; r) @*/
107 /*@ strcpy(a, b) =
108     REQ  TRUE
109     ENS (ex r : TRUE ; strcpy(b) ; ( _ )^* ; r) @*/
110 /*@ memcpy(a, b, C) =
111     REQ  TRUE
112     ENS (ex r : TRUE ; memcpy(a, b) ; ( _ )^* ; r) @*/
113 /*@ return(t) =
114     REQ  TRUE
115     ENS (: TRUE ; return() ; ( _ )^* ; t; -1) @*/

```